Paragraph 1 — Document Chunking

Document chunking is the process of splitting a large document into smaller, manageable pieces before generating embeddings. Chunking improves retrieval quality because embedding models perform better on shorter passages than on entire documents. Common chunking strategies include fixed-size chunking, recursive character splitting, sentence-based chunking, and semantic chunking. The choice of chunk size and overlap has a significant impact on retrieval accuracy and downstream answer quality.

Paragraph 2 — Embeddings

Embeddings are dense numerical vectors that represent the semantic meaning of text. Similar pieces of text produce vectors that are close together in vector space. Modern embedding models can capture contextual meaning beyond simple keyword matching, enabling semantic search. Embeddings are generated once during document ingestion and reused during retrieval, making search both efficient and scalable.

Paragraph 3 — Vector Databases

A vector database stores embeddings and allows efficient similarity search. Instead of searching text directly, it compares vector representations using metrics such as cosine similarity, Euclidean distance, or inner product. Popular vector databases include Qdrant, Pinecone, Weaviate, Milvus, and PostgreSQL with the pgvector extension. Vector databases are a fundamental component of Retrieval-Augmented Generation systems.

Paragraph 4 — Retrieval

Retrieval is the process of identifying the most relevant document chunks for a user's query. A query is converted into an embedding using the same embedding model used during ingestion. The vector database compares this query embedding with stored document embeddings and returns the most similar chunks. These retrieved chunks provide factual grounding for the language model's response.

Paragraph 5 — Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly abbreviated as RAG, combines information retrieval with large language models. Instead of relying solely on the model's internal knowledge, RAG first retrieves relevant context from external documents. The retrieved context is inserted into the model's prompt, allowing the language model to generate answers that are grounded in the uploaded documents. This approach reduces hallucinations and enables responses based on private or domain-specific information.

Paragraph 6 — Prompt Engineering

Prompt engineering is the practice of designing effective instructions for language models. In a RAG system, prompts usually include a system message, retrieved document context, and the user's question. A well-designed prompt instructs the model to answer only from the provided context and to avoid inventing information when the answer cannot be found. Clear prompts improve reliability and consistency.

Paragraph 7 — Similarity Search

Similarity search finds documents that are semantically related to a

query rather than relying only on exact keyword matches. Algorithms such as Approximate Nearest Neighbor (ANN) make similarity search fast even for millions of vectors. The quality of similarity search depends on both the embedding model and the indexing algorithm used by the vector database.

Paragraph 8 — Metadata Filtering

Metadata filtering narrows retrieval results before similarity search is performed. Each document chunk can store metadata such as document ID, page number, author, upload date, or user ID. Applying metadata filters ensures that retrieval searches only within authorized documents or specific subsets of data. This is particularly important in multi-user applications where document isolation is required.

Paragraph 9 — Large Language Models

Large Language Models (LLMs) are transformer-based neural networks trained on massive text corpora. They generate text by predicting the next token in a sequence. While LLMs possess broad general knowledge, they may hallucinate facts when answering domain-specific questions. Integrating an LLM with a retrieval system provides external knowledge that improves factual accuracy and keeps responses grounded in user-provided documents.

Paragraph 10 — End-to-End RAG Pipeline

An end-to-end RAG pipeline typically follows a structured sequence of operations. First, a user uploads a document. The document is then parsed to extract text, which is divided into chunks. Each chunk is converted into an embedding and stored in a vector database along with

metadata. When a user asks a question, the query is embedded and used to retrieve the most relevant chunks. Those chunks are combined into a prompt and passed to a large language model, which generates the final answer. This modular architecture enables scalable, maintainable, and production-ready AI applications.